

智能搜索与问答实验一：命名实体识别

樊建琦

Abstract—Name Entity Recognition(NER) 是对文本中具有特定意义或指代性强的实体进行标注，一般用实体边界识别或确定实体类型实现。此次任务用双向 LSTM 提取每个单词的特征，并通过全连接层映射到标签空间 (B,I,O,START_TAG,STOP_TAG)。每一轮 LSTM 提取出来的特征维护全局的 transition 矩阵，在训练过程中，利用这个矩阵，比较前向算法得分与真实标签路径得分，计算损失函数通过一轮梯度下降找到最优解。推理时使用 Viterbi 算法找出最优标签序列。

1. Introduction

如 Fig. 1所示，此次实验的 BaseLine 是双头 LSTM+CRF。

我们先用 LSTM 得到每个单词的特征 (feat)，feat 是一个 5 维的向量，第 i 个维度代表这个单词转换到第 i 个 tag 的概率 (emmission score)，一个句子的所有单词特征向量拼起来组成 feats，其维度是 (seq_len, tag_size)，如果批量处理句子的话会增加一个维度，控制 batch_size，但是这次试验统一用的一批处理一个句子。

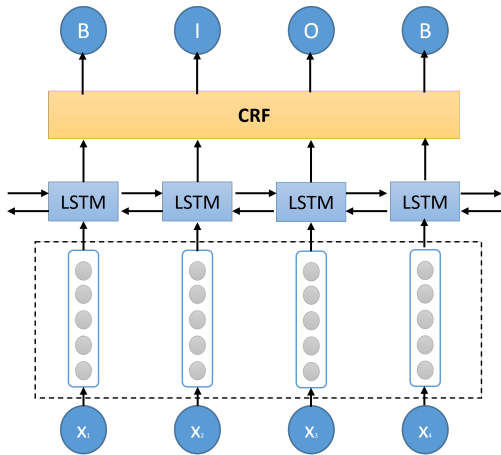


Figure 1. baseline for lab1

利用双头 LSTM 得到每个单词的 emmission score 之后，我们用 CRF 维护全局的转移矩阵 (transitions)。transitions 刻画了一个 5×5 的邻接矩阵，对于每一个 tag 都有转移到所有 tag 的 score(trans_score)。

算法的核心就是将前向变量、转移分数和发射分数相加，计算从任何上一个标签转移到当前标签 next_tag 的总分数，代入公式1，计算所有可能路径的对数分数，添加转移到 STOP_TAG 的分数，计算所有可能以 STOP_TAG 结尾的路径分数的对数和，结束以配分函数的对数值，即所有可能标签序列的分数总和的对数。

$$\log \sum_i \exp(x_i) = \log \sum_i \exp(x_i * c + c) \quad (1)$$

举个例子来说：假设我们有句子“我爱北京天安门”，当我们处理到“北京”这个词时：forward_var 包含了“我爱”这两个词所有可能标签组合的分数 trans_score 表示从“爱”的各个可能标签转移到“北京”的各个可能标签的分数 emit_score 表示“北京”这个词被标注为各个标签的发射分数如果没有 forward_var，我们就只能考虑“爱”和“北京”之间的关系，无法利用“我”的信息。这就是动态规划的核心思想——利用之前计算的结果（子问题

的解）来计算当前问题的解。在 CRF 中，forward_var 就是记忆之前所有计算结果的变量，确保我们在计算当前标签概率时，能够考虑整个序列的前面部分。所以，next_tag_var = forward_var + trans_score + emit_score 实际上是在计算：从序列开始到当前词的所有可能路径的完整分数。

2. Conditional Random Field

设 $X = (x_1, x_2, \dots, x_n)$, $Y = (y_1, y_2, \dots, y_n)$ ，为线性链表示的随机变量序列，给定 X 的条件下， Y 的条件概率分布构成条件随机场（即：满足马尔科夫性），称 $P(Y|X)$ 为线性链条件随机场。

CRF 的引入很大程度上是因为我们没有有效的办法处理状态间的转移问题，比如我们此刻知道“我们爱北京”的每一个单词的隐藏层向量，或者说特征向量。但是此刻我们缺乏一个可行的手段表征状态间转移的问题。

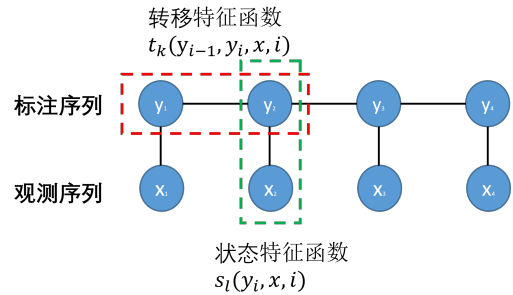


Figure 2. 状态特征函数

在序列标注任务中，标签之间存在强烈的依赖关系。例如，在命名实体识别 (NER) 任务中，标签“**I-LOC**”（地点内部）几乎不可能出现在标签“**B-PER**”（人名开始）之后。这种标签间的转移约束是序列标注任务的核心特征。传统的神经网络模型（如 BiLSTM）虽然能够捕获输入序列的上下文信息，但在建模标签之间的依赖关系方面存在局限性。

CRF 层作为序列标注模型的输出层，可以有效解决这一问题。它通过引入转移矩阵 (transition matrix) 来显式地建模相邻标签之间的转移概率。在我们的实现中，转移矩阵 T 定义为一个大小为 $|S| \times |S|$ 的矩阵，其中 S 是标签集合。矩阵元素 T_{ij} 表示从标签 j 转移到标签 i 的评分（非概率值）。

形式化地，给定一个输入序列 $X = (x_1, x_2, \dots, x_n)$ 和对应的标签序列 $Y = (y_1, y_2, \dots, y_n)$ ，CRF 定义了一个全局评分函数：

$$s(X, Y) = \sum_{i=1}^n P_{i, y_i} + \sum_{i=2}^n T_{y_i, y_{i-1}} + T_{y_1, \text{START}} + T_{\text{STOP}, y_n} \quad (2)$$

- P_{i, y_i} 是 BiLSTM 为第 i 个位置的词分配标签 y_i 的发射评分
- $T_{y_i, y_{i-1}}$ 是从标签 y_{i-1} 转移到 y_i 的转移评分
- $T_{y_1, \text{START}}$ 和 T_{STOP, y_n} 分别是序列开始和结束的特殊转移评分

我们的模型使用对数线性 CRF，将条件概率定义为：

$$P(Y|X) = \frac{\exp(s(X, Y))}{\sum_{Y'} \exp(s(X, Y'))} \quad (3)$$

其中分母是所有可能标签序列的评分之和，通常称为配分函数 (partition function)。由于可能的标签序列数量随序列长度呈指数增长，直接计算配分函数是不可行的。因此，我们采用动态规划算法高效计算。

在实现中，我们使用前向算法 (forward algorithm) 和维特比算法 (Viterbi algorithm) 分别用于训练和解码：

1. **forward()**: 计算配分函数 $\sum_{Y'} \exp(s(X, Y'))$ ，这在我们代码中由 `_forward_alg` 函数实现。该算法利用动态规划原理，递归地计算到达每个位置的每个标签的所有可能路径的评分总和。
2. **__viterbi_decode()**: 找到给定输入序列的最可能标签序列，即 $\arg \max_Y s(X, Y)$ ，这在我们代码中由 `_viterbi_decode` 函数实现。该算法同样使用动态规划，但只保留到达每个状态的最高评分路径。

我们的损失函数被定义为负对数似然：

$$\mathcal{L} = -\log P(Y|X) = -s(X, Y) + \log \sum_{Y'} \exp(s(X, Y')) \quad (4)$$

这对应于代码中的 `neg_log_likelihood` 函数，它计算 `forward_score` 与 `gold_score` 之差。通过最小化这个损失函数，我们既优化了 BiLSTM 的参数（通过发射评分），也优化了 CRF 层的转移矩阵。

在我们的实现中，CRF 层带来了几个关键优势：

1. **tag_scheme**: CRF 显式地建模标签间的依赖关系，使预测结果符合语言学规则（如 “I-LOC” 不能跟在 “B-PER” 之后）。
2. **global optimization**: CRF 考虑整个序列的标签分配，而不是独立地为每个位置分配标签，从而产生全局一致的预测结果。

3. LSTM

3.1. Necessity for Bi-LSTM

在序列标注任务中，标注一个词不仅需要考虑其前面的词，还需要考虑后面的词。例如，在命名实体识别中，要判断 “北京” 是否为地名，不仅需要看前面的词（如 “去”、“到” 等），也需要看后面的词（如 “市”、“大学” 等）。单向 LSTM 只能捕获单向上下文信息，这对序列标注任务是不足的。

因此，我们采用双向 LSTM (Bidirectional LSTM, BiLSTM) 结构：

$$\text{bidirectional} = \text{True} \quad (5)$$

BiLSTM 包含两个方向的 LSTM：前向 LSTM 处理从左到右的序列，后向 LSTM 处理从右到左的序列。对于序列中的每个位置，我们将两个方向的隐藏状态连接起来，得到一个能够同时编码前向和后向上下文的表示。这在代码中体现为：

$$\text{hidden_dim} // 2 \quad (6)$$

每个方向的 LSTM 输出维度为总隐藏维度的一半，连接后恢复为完整的隐藏维度。

3.2. __get_lstm_features()

```

1  def __get_lstm_features(self, sentence):
2      self.hidden = self.init_hidden()
3
4      # (sentence_length, 1, embedding_dim), 1
5      ↪ because model deals with one sentence at a time
6      embeds = self.word_embeddings(sentence).view(len(
7      ↪ sentence), 1, -1)
8
9      # (sentence_length, hidden_dim), so every word
10     ↪ has their own hidden state vector
11     lstm_out, self.hidden = self.lstm(embeds, self.
12     ↪ hidden)
13     lstm_out = lstm_out.view(len(sentence), self.
14     ↪ hidden_dim)
15
16     # (sentence_length, tagset_size), so every word
17     ↪ has their own tag score
18     lstm_feats = self.hidden2tag(lstm_out)
19     return lstm_feats

```

Code 1. `__get_lstm_features()`

在我们的 BiLSTM-CRF 模型中，BiLSTM 扮演特征提取器的角色。首先，输入序列通过嵌入层转换为密集向量表示，然后，这些嵌入向量被送入 BiLSTM 进行处理，BiLSTM 的输出 `lstm_out` 是一个形状为 `[sequence_length, hidden_dim]` 的张量，每一行代表序列中对应位置的上下文化表示。这些表示被进一步映射到标签空间，其中，`hidden2tag` 是一个线性变换，这样，我们得到了每个位置上各个标签的评分，也就是 CRF 层的发射分数 (emission scores)。

3.3. WHY BiLSTM

在我们的序列标注模型中，BiLSTM 带来了以下几个关键优势：

1. **Contextual Representation**: BiLSTM 能够为每个词生成上下文敏感表示，这对于区分同形异义词（如 “苹果” 可以是水果或公司名）至关重要。
2. **Long-range Dependency**: 与传统 RNN 相比，LSTM 能够捕获更长距离的依赖关系，这在处理长句子时尤为重要。
3. **Bidirectional Context**: 双向结构确保模型能够同时利用左侧和右侧的上下文信息，提供更全面的语境理解。
4. **Feature Engineering Reduction**: BiLSTM 能够自动学习有效的特征表示，减少了手工特征工程的需要。
5. **Parameter Efficiency**: 通过参数共享，BiLSTM 能够高效地处理不同长度的序列，避免了固定窗口方法的局限性。

3.4. BiLSTM PLUS CRF

将 BiLSTM 与 CRF 结合是一种合理且有效的设计选择，原因如下：

1. **Complementary Strengths**: BiLSTM 擅长学习单词的上下文表示，而 CRF 擅长建模标签之间的转移约束。这两个组件互为补充，形成了一个功能更强大的整体。
2. **End-to-end Training**: BiLSTM-CRF 架构允许端到端训练，所有参数（包括词嵌入、LSTM 权重和 CRF 转移矩阵）

都可以通过反向传播联合优化。

3. **Structured Prediction**: 序列标注本质上是一个结构化预测问题, 需要考虑标签序列的整体一致性。BiLSTM 提供特征, CRF 确保预测的结构合理性。

4. **Empirical Success**: BiLSTM-CRF 架构在多种序列标注任务上取得了显著成功, 成为这类任务的标准解决方案之一。

在我们的实现中, BiLSTM 的输出 (经过线性变换后) 作为 CRF 层的输入:

```
feats = self._get_lstm_features(sentence) (7)
```

然后, CRF 层使用这些特征计算最可能的标签序列:

```
score, tag_seq = self._viterbi_decode(lstm_feats) (8)
```

这种架构设计使我们的模型能够同时利用 BiLSTM 的特征学习能力和 CRF 的结构化预测能力。

4. Function Analysis

这次实验有三个核心算法: CRF 的前向传播以及用作 decoder 的 viterbi 算法, 本章先后对这两个算法进行详细的解释。

4.1. _forward_alg()

首先, 我们从例子来入手这个算法:

假设我们有句子“我爱北京”, 标签集为 O, B-LOC, I-LOC, START, STOP 初始化时, forward_var = [0, -10000, -10000, -10000, -10000] (START_TAG 的位置为 0)。

对于第一个词“我”: 计算从 START_TAG 转移到每个可能标签的分数更新 forward_var, 例如可能变为 [-2.3, -5.1, -9.8, -10000, -10000] 对每个后续词重复这个过程最后加上转移到 STOP_TAG 的分数, 得到最终的配分函数值。

这个函数的核心目的是计算所有可能标签序列的分数和的对数, 用于 CRF 的训练过程。这个值与正确标签序列的分数之差构成了损失函数。

接下来我们对 CRF 的前向传播算法做详细的解释:

```
1 def _forward_alg(self, feats):
2     # Do the forward algorithm to compute the
3     ↪ partition function
4     init_alphas = torch.full((1, self.tagset_size),
5     ↪ -10000.)
6     # START_TAG has all of the score.
7     init_alphas[0][self.tag_to_ix[START_TAG]] = 0.
8
9     # Wrap in a variable so that we will get
10    ↪ automatic backprop
11    forward_var = init_alphas
12
13    """Iterate through the sentence.
14    in the example sentence, there're 11 words, so
15    ↪ the feats.size() = [11,5]
16    """
17    for feat in feats:
18        alphas_t = [] # The forward tensors at this
19        ↪ timestep
```

```
15         for next_tag in range(self.tagset_size):
16             # broadcast the emission score: it is
17             ↪ the same regardless of
18             # the previous tag
19             emit_score = feat[next_tag].view(
20                 1, -1).expand(1, self.tagset_size)
21             # the ith entry of trans_score is the
22             ↪ score of transitioning to
23             # next_tag from i
24             trans_score = self.transitions[next_tag]
25             ↪ ].view(1, -1)
26             # The ith entry of next_tag_var is the
27             ↪ value for the
28             # edge (i -> next_tag) before we do log-
29             ↪ sum-exp
30             next_tag_var = forward_var + trans_score
31             ↪ + emit_score
32             # The forward variable for this tag is
33             ↪ log-sum-exp of all the
34             # scores.
35             alphas_t.append(log_sum_exp(next_tag_var
36             ↪ ).view(1))
37             forward_var = torch.cat(alphas_t).view(1, -1)
38             ↪ )
39             terminal_var = forward_var + self.transitions[
40             ↪ self.tag_to_ix[STOP_TAG]]
41             alpha = log_sum_exp(terminal_var)
42             return alpha
```

Code 2. _forward_alg()

算法的核心是两个矩阵: self.transitions 和 feats。转移矩阵是全局的, shape 是 (5,5), 表征一个 tag 转换到另一个 tag 的 score; feats 是 sentence 中所有 feat 拼起来的结果, 每一个 feat 的 shape 是 (5,), 表征转换到那个 tag 的 score。

alphas 在初始化的时候仅开放了 start 的权限 (对应 init_alphas[0][3]), 其他的设置为不可达, 此为开始状态节点。forward_var 被赋值为 init_alphas, 因为后面将用 forward_var 代表前序所有历史经验做加法。

从这两个大的 for 循环可以看出来, 算法是在处理一个 seq_len 行, tag_size 列的大表。不妨我们从任意“一行”开始推导, 我们可以看到放进 log_sum_exp 的是一个加法的产物, 这个东西由三个部分组成: emit_score, trans_score 以及 forward_var。emit_score 是自己这个单词而言, 转换到目前这个列 (称作 next tag) 的 score, 之后把他延展到 5 的宽度 (方便加法); trans_score 是任何能够转换到 next tag 的 score (len 应该正好是 5, 因为 5 个 tag 都能转换到这里)。

然后我们使用 log_sum_exp 函数对所有可能的路径进行对数空间下的求和, 将结果存入 alphas_t。完成内层循环后, 将 alphas_t 中的所有得分合并为新的 forward_var, 继续处理下一个词。最后, 我们考虑转移到 STOP_TAG 的得分, 计算 terminal_var, 并对其应用 log_sum_exp 得到最终的 alpha 值, 这就是所有可能路径的总得分 (对数空间下)。

4.2. _viterbi_decode()

维特比算法是序列标注问题中寻找最优标签序列的经典算法。继续使用上面的例子来解释这个过程:

假设我们有句子” 我爱北京”，标签集为 O, B-LOC, I-LOC, START, STOP 初始化时, `init_vvars = [0, -10000, -10000, -10000, -10000]` (`START_TAG` 的位置为 0)。

对于每个词和可能的标签，维特比算法与前向算法的关键区别在于：前向算法计算所有可能路径的总和（使用 `log_sum_exp`）维特比算法只保留每个状态的最佳前导路径（使用 `argmax`）

对于第一个词” 我”：计算从 `START_TAG` 转移到每个可能标签的分数但不同于前向算法，这里只保留最高分数的前导标签记录每个标签的最优前导标签（backpointer）更新 `forward_var`，只保留最佳路径的分数。

```

36     for bptrs_t in reversed(backpointers):
37         best_tag_id = bptrs_t[best_tag_id]
38         best_path.append(best_tag_id)
39         # Pop off the start tag (we dont want to return
    ↳ that to the caller)
40         start = best_path.pop()
41         assert start == self.tag_to_ix[START_TAG] #
    ↳ Sanity check
42         best_path.reverse()
43         return path_score, best_path

```

Code 3. Viterbi 解码算法的 Python 实现.

首先，算法初始化 `backpointers` 列表来存储每一步的最佳前导标签。与前向算法类似，初始化 `init_vvars` 矩阵，仅 `START_TAG` 对应位置的值为 0，其余为-10000。

在两个嵌套循环中，算法为当前词的每个可能标签计算最佳得分路径：`next_tag_var = forward_var + self.transitions[next_tag]` 计算从前一步的所有标签转移到当前标签的分数使用 `argmax` 找出最佳前导标签的索引将这个最佳前导标签索引存入 `bptrs_t`（用于后向追踪）将最高分数存入 `viterbivars_t`

内层循环结束后，加入发射分数 `feat`，更新 `forward_var` 为当前步的最佳路径分数，并将 `backpointers` 添加到总列表中。

处理完所有词后，考虑到 `STOP_TAG` 的转移，计算最终得分和最后一个最优标签。

最后，通过回溯 `backpointers` 列表构建最优路径：从最后一个最优标签开始使用存储的 `backpointers` 逐步回溯反转路径并移除 `START_TAG` 返回最优路径得分和标签序列

维特比算法的核心思想是”贪心地”保留每个时间步和每个标签的最优路径，从而避免了计算所有可能路径的组合爆炸问题，复杂度从指数级降低到线性级。

4.3. neg_log_likelihood()

负对数似然函数是 CRF 模型训练过程中的核心损失函数。该函数计算模型预测的所有可能路径的概率分布与真实标签路径之间的差异，用于模型参数的优化。

继续使用前面的例子解释：对于句子” 我爱北京” 和真实标签序列 O, O, B-LOC, I-LOC，我们需要：1. 计算所有可能的标签序列的总分数（配分函数）2. 计算真实标签序列的分数 3. 两者之差即为损失函数值

```

1     def neg_log_likelihood(self, sentence, tags):
2         feats = self._get_lstm_features(sentence)
3         forward_score = self._forward_alg(feats) #现
    ↳ 在提取出来的特征矩阵跑出来的得分
4         gold_score = self._score_sentence(feats, tags)
    ↳ #正确路径跑出来的得分
5         return forward_score - gold_score

```

Code 4. CRF 负对数似然损失函数的实现.

这个函数虽然代码简短，但包含了 CRF 模型核心的训练思想。它由三个关键步骤组成：

首先，通过调用 ‘`_get_lstm_features(sentence)`’ 从 BiLSTM 网络获取每个词的特征表示（发射分数）。这一步将输入句子转换

```

1     def _viterbi_decode(self, feats):
2         backpointers = []
3
4         # Initialize the viterbi variables in log space
5         init_vvars = torch.full((1, self.tagset_size), -
    ↳ 10000.)
6         init_vvars[0][self.tag_to_ix[START_TAG]] = 0
7
8         # forward_var at step i holds the viterbi
    ↳ variables for step i-1
9         forward_var = init_vvars
10        for feat in feats:
11            bptrs_t = [] # holds the backpointers for
    ↳ this step
12            viterbivars_t = [] # holds the viterbi
    ↳ variables for this step
13
14            for next_tag in range(self.tagset_size):
15                # next_tag_var[i] holds the viterbi
    ↳ variable for tag i at the
16                # previous step, plus the score of
    ↳ transitioning
17                # from tag i to next_tag.
18                # We don't include the emission scores
    ↳ here because the max
19                # does not depend on them (we add them
    ↳ in below)
20                next_tag_var = forward_var + self.
    ↳ transitions[next_tag]
21                best_tag_id = argmax(next_tag_var)
22                bptrs_t.append(best_tag_id)
23                viterbivars_t.append(next_tag_var[0][
    ↳ best_tag_id].view(1))
24                # Now add in the emission scores, and assign
    ↳ forward_var to the set
25                # of viterbi variables we just computed
26                forward_var = (torch.cat(viterbivars_t) +
    ↳ feat).view(1, -1)
27                backpointers.append(bptrs_t)
28
29            # Transition to STOP_TAG
30            terminal_var = forward_var + self.transitions[
    ↳ self.tag_to_ix[STOP_TAG]]
31            best_tag_id = argmax(terminal_var)
32            path_score = terminal_var[0][best_tag_id]
33
34            # Follow the back pointers to decode the best
    ↳ path.
35            best_path = [best_tag_id]

```

为深度特征，为后续的序列标注提供基础。

然后，函数计算两个关键分数：1. ‘forward_score’：通过调用 ‘_forward_alg(feats)’ 计算所有可能标签序列的对数和。这个分数代表了模型当前对所有可能路径的预测分布。2. ‘gold_score’：通过调用 ‘_score_sentence(feats, tags)’ 计算真实标签序列的分数。这个分数表示正确路径在当前模型下的评分。

最后，返回这两个分数的差值 ‘forward_score - gold_score’，即为负对数似然损失。

从概率角度理解，这个损失函数可以表示为：
$$* \log \frac{e^{\text{score}(X,Y)}}{\sum_{Y'} e^{\text{score}(X,Y')}} = * \text{score}(X,Y) + \log \sum_{Y'} e^{\text{score}(X,Y')}$$

其中， X 是输入句子， Y 是真实标签序列， Y' 表示所有可能的标签序列。

这个损失函数的优化目标是：提高正确标签序列的分数，同时降低所有可能序列的总分数。通过梯度下降最小化这个损失，模型将逐渐学习到更准确的参数，使得正确标签序列的概率接近 1，其他错误序列的概率接近 0。

在实际训练中，‘_score_sentence’ 和 ‘_forward_alg’ 函数共享相同的特征矩阵和转移矩阵，确保了损失函数的一致性和可微性，从而可以通过反向传播有效地更新模型参数。

4.4. _score_sentence()

这个函数计算给定标签序列的分数，是 CRF 模型中评估特定路径得分的关键组件。与前向算法不同，它不考虑所有可能的路径，而是计算一条特定路径（通常是真实标签序列）的得分。

继续使用前面的例子进行解释：对于句子“我爱北京”和真实标签序列 O, O, B-LOC, I-LOC，我们需要计算这个特定序列的总分数。

```
1 def _score_sentence(self, feats, tags):
2     # Gives the score of a provided tag sequence
3     score = torch.zeros(1)
4     tags = torch.cat([torch.tensor([self.tag_to_ix[
5         ↪ START_TAG]], dtype=torch.long), tags])
6     for i, feat in enumerate(feats):
7         score = score + \
8             self.transitions[tags[i + 1], tags[i]] +
9         ↪ feat[tags[i + 1]]
10    score = score + self.transitions[self.tag_to_ix[
11    ↪ STOP_TAG], tags[-1]]
12    return score
```

Code 5. 计算特定标签序列得分的 Python 实现。

这个函数的实现非常直观，它直接计算一个给定标签序列的得分。函数接收两个关键参数：1. ‘feats’：从 BiLSTM 网络获取的特征矩阵，维度为 (句子长度, 标签集大小) 2. ‘tags’：真实的标签序列，维度为 (句子长度)

函数首先初始化得分为 0，然后将 START_TAG 添加到标签序列的开头，以便处理从开始状态的转移。

之后，函数在一个循环中遍历句子中的每个位置，累加三种得分：1. 当前位置的词转换为对应标签的发射得分：‘feat[tags[i + 1]]’ 2. 从前一个标签到当前标签的转移得分：‘self.transitions[tags[i + 1], tags[i]]’ 3. 最后加上从最后一个标签转移到 STOP_TAG 的得分

与 _forward_alg() 函数相比，_score_sentence() 的计算更为简单直接：- _forward_alg()：计算所有可能路径的对数和（使用动态规划）- _score_sentence()：仅计算一条特定路径的得分（简单的累加）

这两个函数配合使用，构成了 CRF 模型的负对数似然损失函数，驱动模型学习正确的参数。在训练过程中，模型会增加正确路径的得分，同时降低错误路径的得分，逐步提高预测准确率。

此函数的简洁实现反映了条件随机场模型的直观性：一个标签序列的得分是由沿途所有局部特征得分的简单累加构成的。这种设计使得 CRF 能够有效地建模序列标注问题中的局部和全局依赖关系。

4.5. forward()

PyTorch 中的 ‘forward()’ 方法是模型前向传播的核心函数，在 BiLSTM-CRF 模型中，它用于模型推理阶段（测试时）获取最优标签序列。这个函数是模型对外的主要接口，当我们调用 ‘model(sentence)’ 时，实际上就是调用了这个函数。

```
1 def forward(self, sentence):
2     """ Tips for forward function:
3         forward function is used to get the best path
4         ↪ and the score of the best path(test time)
5         _forward_alg is used to get the score of the
6         ↪ sentence(train time)
7     """
8     # dont confuse this with _forward_alg above.
9     #for example the sentence was like tensor([[1, 2
10    ↪ , 3, 4, 5, 6, 7, 8, 9, 10]])
11    # Get the emission scores from the BiLSTM
12    lstm_feats = self._get_lstm_features(sentence)
13
14    # Find the best path, given the features.
15    score, tag_seq = self._viterbi_decode(lstm_feats
16    ↪ )
17    return score, tag_seq
```

Code 6. 模型前向传播函数的 Python 实现。

‘forward()’ 函数的实现非常简洁明了，它主要完成两个步骤：

1. 调用 ‘_get_lstm_features(sentence)’ 获取 BiLSTM 提取的特征
2. 调用 ‘_viterbi_decode(lstm_feats)’ 使用维特比算法找出最优标签序列

然后返回最优路径的得分和对应的标签序列。

4.6. forward() 与 _forward_alg() 的对比分析

尽管名称相似，如 Table 1 所示，‘forward()’ 和 ‘_forward_alg()’ 在功能和用途上有着本质的区别：

Table 1. Comparison with forward() and _forward_alg()

Aspect	forward()	_forward_alg()
主要用途	test and inference	train
计算目标	寻找最优标签序列	可能路径的对数和
算法核心	viterbi	Forward
计算策略	选择每步最大值	计算所有路径的和
返回结果	路径得分和标签序列	配分函数的对数值
在 CRF 中的角色	decoder	损失函数组件

‘forward()’函数是模型的核心推理接口，用于在给定输入句子的情况下，预测最可能的标签序列。它采用了著名的维特比算法，这是一种动态规划方法，能够高效地找出最优路径。在实际应用中，当我们部署模型进行预测时，就是调用这个函数。

而 ‘_forward_alg()’则是训练阶段的关键组件，用于计算 CRF 的配分函数（归一化因子）。配分函数表示所有可能标签序列的得分之和，是计算条件概率 $P(Y|X)$ 的分母部分。这个函数使用前向算法，同样是动态规划的应用，但目标是计算所有路径的和，而不是找出最优路径。

从数学角度看，两者的区别可以表示为：- ‘_forward_alg()’: 计算 $\log \sum_Y e^{\text{score}(X,Y)}$ （所有可能路径的对数和）- ‘forward()’: 寻找 $\arg \max_Y \text{score}(X,Y)$ （得分最高的路径）

从算法复杂度来看，两者都是 $O(nT^2)$ 的时间复杂度，其中 n 是序列长度， T 是标签集大小。但它们的操作却截然不同：一个使用对数求和操作(log-sum-exp)，另一个使用最大值操作(argmax)。

这种设计反映了 CRF 模型的优雅之处：在训练时考虑所有可能的路径以获得全局最优解，而在推理时只需找出一条最优路径即可。这种训练与推理的分离使得 CRF 在保持模型复杂性的同时，还能保证推理过程的高效性。

4.7. In-Depth Analysis of the log_sum_exp() Function

```

1 def log_sum_exp(vec):
2     max_score = vec[0, argmax(vec)]
3     max_score_broadcast = max_score.view(1, -1).expand(1
4         ↪ , vec.size()[1])
5     return max_score + \
6         torch.log(torch.sum(torch.exp(vec -
7             ↪ max_score_broadcast)))

```

Code 7. log_sum_exp 函数的 Python 实现。

4.7.1. Mathematical Principle

log_sum_exp 函数是计算条件随机场（CRF）模型中关键的数值稳定性函数，其数学表达式为：

$$\text{LogSumExp}(x_1, x_2, \dots, x_n) = \log \left(\sum_{i=1}^n e^{x_i} \right) \quad (9)$$

这个函数看似简单，但在 CRF 模型中扮演着至关重要的角色，尤其是在前向算法（Forward Algorithm）的实现中。为什么我们需要这个特殊的函数？这涉及到两个关键问题：数值溢出和计算效率。

直接计算 $\sum_{i=1}^n e^{x_i}$ 可能导致数值溢出，因为指数函数 e^x 对于较大的 x 值会快速增长到超出浮点数表示范围。而当 x 值很小时，又可能导致下溢问题，即计算结果太小以至于被近似为零。为了解决这个问题，我们使用一个数学技巧：

$$\log \left(\sum_{i=1}^n e^{x_i} \right) = \log \left(e^{x_{\max}} \cdot \sum_{i=1}^n \frac{e^{x_i}}{e^{x_{\max}}} \right) = x_{\max} + \log \left(\sum_{i=1}^n e^{x_i - x_{\max}} \right)$$

其中 $x_{\max} = \max(x_1, x_2, \dots, x_n)$ 。这样，指数项 $e^{x_i - x_{\max}}$ 的最大值为 1（当 $x_i = x_{\max}$ 时），有效避免了溢出问题。

4.7.2. CODE IMPLEMENTATION

我们逐行分析函数实现：

1. `max_score = vec[0, argmax(vec)]`: 获取向量中的最大值。这里使用了自定义的 `argmax` 函数找到最大值的索引，然后提取该最大值。
2. `max_score_broadcast = max_score.view(1, -1).expand(1, -1).expand(1, vec.size()[1])`: 将最大值扩展成与原向量相同形状的张量，便于后续的向量化操作。
3. `return max_score + torch.log(torch.sum(torch.exp(vec - max_score_broadcast)))`: 实现上述数学公式，先减去最大值，计算指数，求和，取对数，最后加回最大值。

4.7.3. CORE ROLE IN CRF

在 BiLSTM-CRF 模型架构中，log_sum_exp 函数主要在前向算法（_forward_alg）中被调用，用于计算所有可能路径的对数和（配分函数的对数值）。让我们看看它在前向算法中的具体应用：

1. **Partition Function Calculation**: CRF 模型中，配分函数 $Z(x)$ 是归一化因子，表示为 $Z(x) = \sum_y \exp(\text{score}(x, y))$ ，其中 y 遍历所有可能的标签序列。由于标签序列数量随句子长度指数增长，直接计算不可行，因此使用动态规划的前向算法，其中 log_sum_exp 是关键步骤。
2. **Application in Forward Recurrence**: 在前向算法的每一步，需要计算到达当前状态的所有可能路径的得分和。具体地，对于每个标签 j 在位置 t ，计算：

$$\alpha_t(j) = \log \sum_{i=1}^n e^{\alpha_{t-1}(i) + w_{ij} + e_j(x_t)} \quad (10)$$

其中 $\alpha_{t-1}(i)$ 是前一位置的前向变量， w_{ij} 是从标签 i 转移到标签 j 的得分， $e_j(x_t)$ 是观测到 x_t 并标记为 j 的发射得分。这正是 log_sum_exp 函数的用武之地。

4.7.4. FOR VITERBI

log_sum_exp 函数在前向算法和维特比算法中的不同应用，反映了 CRF 中训练与推理的本质区别：

- **Forward Algorithm**: 使用 log_sum_exp 计算所有路径的和，对应于求解 $\log \sum_y \exp(\text{score}(x, y))$
- **Viterbi Algorithm**: 使用 argmax 找出最佳路径，对应于求解 $\arg \max_y \text{score}(x, y)$

这种区别体现在代码中：前向算法使用 log_sum_exp，而维特比算法使用 argmax 函数。

4.7.5. STABILITY

`log_sum_exp` 函数对模型训练的数值稳定性贡献巨大：

- **Preventing Overflow**: 在处理长序列或大型标签集时，得分可能达到很大的数值，直接计算 e^{score} 会导致溢出。`log_sum_exp` 通过减去最大值再计算指数，有效防止了这种情况。
- **Maintaining Precision**: 即使在极端情况下（如某些路径得分远高于其他路径），`log_sum_exp` 也能保持计算精度，确保梯度信息不会因数值问题而丢失。
- **Accelerating Training**: 通过避免数值不稳定导致的 NaN 或 Inf 值，`log_sum_exp` 使训练过程更加稳定和高效，减少了因数值问题导致的训练中断或失败。

4.7.6. BROADER SIGNIFICANCE

`log_sum_exp` 不仅仅是一个技术细节，它反映了深度学习中软性决策与硬性决策的区别：

- **Soft Decision**: 考虑所有可能性并根据其概率权衡（使用 `log_sum_exp`）
- **Hard Decision**: 只选择最可能的结果（使用 `argmax`）

在 CRF 模型的训练阶段，我们优化所有可能路径的概率分布，而在推理阶段，只需要找出最可能的一条路径。这种“训练时软性、推理时硬性”的范式不仅适用于 CRF，也普遍存在于其他机器学习模型中。

5. 思考题

5.1. 传统机器学习 CRF 方法实现

列举出所有可能的路径并计算得分，效率问题？有没有更好的方法？

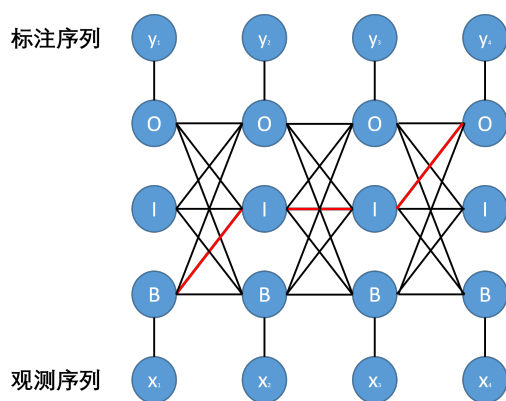


Figure 3. 穷举图

5.1.1. 列举出所有可能的路径并计算得分

标注长度为 3 (分别是 B, I, O)，序列长度为 4 (这里指的是 sentence 的长度)，因此共有 3^4 条路径。

5.1.2. 效率问题

如果用上面的方法的话，可能路径会以指数形式增长，所以用 **Forward-Backward algorithm** 和 **Viterbi algorithm**，以动态规划的形式大规模减少计算的复杂度。

5.2. 为什么结合 LSTM 和 CRF

使用 LSTM 做特征提取，虽然 LSTM 可以直接预测每个 token 的标注标签概率，但因每个 token 标签的预测是相互独立的，无法学习到标注标签间的转移依赖，因此结合 CRF 使用。